



신혜민

FRONTEND DEVELOPER

CONTACT

shinhm21@gmail.com

github.com/hm1n

[기술 블로그](#)

EDUCATION

2018.03 - 2024.02

미디어커뮤니케이션과,
컴퓨터공학과 졸업

세종대학교

INTERNSHIP

2022.07 - 2022.12

CJ올리브네트웍스
디지털러닝파트

CJ그룹 임직원 교육 플랫폼
백오피스 기획 및 운영

CERTIFICATE

2026.03

OPIc IH

2022.02

컴퓨터활용능력 1급

개발, 배포, 실제 사용자 대상 피드백 수집과 개선까지

서비스의 End-to-End 사이클을 경험한 신입 프론트엔드 개발자입니다.

사용자 피드백을 기반으로 서비스를 개선합니다. 실사용자 51명을 대상으로 만족도 조사를 진행해 정산 현황 관리와 채팅 기능에 대한 개선 과제를 도출했고 이를 실제 기능에 반영해 현재까지 서비스를 운영 및 개선하고 있습니다.

실제 지표를 측정하고 문제를 파악합니다. 채팅 응답이 느린 문제를 파악하고 지연 시간을 직접 측정해 실제 지연이 약 3.9초임을 확인하고, 채팅 구독 로직을 개선하고 낙관적 업데이트를 적용해 지연 시간을 0.05초로 줄였습니다.

또한 Search Console에서 색인 콘텐츠가 없다는 문제를 발견해 랜딩페이지와 FAQ 영역을 새로 기획하고 구현해서 일평균 검색 노출을 4.6배 늘렸습니다.

구현에서 끝내지 않고 배포 이후 운영까지 고려합니다. EAS 기반 CI/CD와 OTA 업데이트 환경을 구축해 월평균 빌드 횟수를 56% 줄이고 빌드 실패 횟수를 5.4회에서 0.5회로 낮춰 안정적으로 서비스를 운영할 수 있는 환경을 만들었습니다.

HOW I WORK WITH AI

저는 AI를 이렇게 활용해요

공동 규칙 문서화 | Claude Code, Codex 등 도구가 달라도 AGENTS.md에 프로젝트 규칙을 먼저 정리해 같은 원칙으로 작업하게 합니다. [\[GitHub README\]](#)

반복 업무 Skill화 | 이슈 작성이나 릴리즈 준비처럼 반복되는 작업은 매번 새로 설명하지 않고, 절차와 기준을 Skill로 만들어 재사용합니다.

릴리즈 파이프라인 자동화 | 릴리즈 노트를 일일이 정리하지 않고 AI Skill로 작성한 뒤 배포가 성공하면 GitHub Actions가 태그와 GitHub Release까지 자동으로 생성합니다.

EXPERIENCE

Wanted AI Native Career Camp

진행 중 | 2026.07 - 2026.11

- AI 시대 취업 트렌드 및 직군별 AI 활용 사례 학습
- 프롬프트 설계, 노코드 자동화, AI 에이전트 구축 등 AI 워크플로우 관련 교육 이수

LG U+ URECA 1기 프론트엔드 교육과정

수료 | 2024.06 - 2024.12

- React, Web, HTML/CSS/JavaScript 등 웹 프론트엔드 개발에 필요한 기초 교육 이수
- 3개 프로젝트에서 모두 FE 팀장을 도맡아 프로젝트 총괄 및 Notion 기반 문서화 기여
- 종합프로젝트에서 프론트엔드 비대면 부문 **6개 팀 중 1위로 최우수상 수상**

PROJECTS

엔빵 / 구독 공유 관리 웹 서비스

2025.02 - 2025.10(FE 4인) / 2026.05 - 현재(개인)

[서비스 바로가기 →](#) | [깃허브 바로가기 →](#)

여러 명이 나눠 쓰는 구독 서비스의 결제일, 각자 낼 금액, 납부 여부를 한 곳에서 관리할 수 있는 구독 공유 관리 서비스입니다.
교육을 통해 얻은 기술력으로 일상에서 느낀 불편을 직접 해결해 보자는 취지로 주제를 제안하고 기획부터 배포까지 주도했습니다.
2025년 2월 팀 프로젝트로 시작해 2026년 5월부터는 개인 프로젝트로 전환해 지금까지 배포 및 운영하고 있습니다.

TECH STACK

Next.js | Supabase | TypeScript | TailwindCSS | Vercel

Next.js | 검색 유입 확보를 위해 크롤러에게 렌더링된 페이지를 제공할 수 있는 SSR 기반 프레임워크를 활용

Supabase | 빠르게 MVP를 검증하고 Realtime, DB 등 백엔드 기능을 BaaS로 해결해 개발 공수를 줄이기 위해 채택

WHAT I DID

• 사용자 피드백 기반 MVP 검증 및 서비스 개선

- 문제 | 구독 서비스를 공유 이용 시 결제일, 정산 금액, 납부 여부를 여러 서비스에서 따로 관리해야 하는 불편 존재
- 해결 | 사용자 조사와 경쟁 서비스 분석을 바탕으로 MVP를 정의하고 구독 그룹 생성, 구성원 초대, 정산 현황 관리 기능 개발
- 결과 | 실제 사용자 51명 대상 만족도 조사에서 **만족도 점수 평균 7.94점**을 확인하고 푸시 알림, 채팅 등 핵심 개선 과제 도출

• AI Skill과 GitHub Actions 기반 워크플로우로 릴리즈 소요 시간 75% 단축

- 문제 | 새 버전 릴리즈마다 릴리즈 브랜치, 이슈, PR 생성 및 merge 등 반복 작업이 필요해 실제 기능 개발과 무관한 작업에 시간이 소요되어 개발 경험 저하
- 해결 | 맥락 판단이 필요한 작업은 AI Skill이, 결과가 결정적인 반복 작업은 GitHub Actions가 담당하도록 역할을 분리해 PR 수집, 릴리즈 노트 작성, 배포 성공 감지 후 태그 및 Release 생성 과정을 자동화하는 워크플로우 구축
- 결과 | 릴리즈 단계를 **5단계 → 2단계**로 줄이고, 평균 소요 시간을 **6분 36초 → 1분 39초**로 **약 75% 단축**

• 채팅 메시지 전송 체감 지연 3.9초 → 0.25초 개선

- 문제 | 메시지를 보낸 뒤 화면에 뜨기까지 약 4초가 걸려 정산 공지와 대화 흐름이 끊기는 문제 발생
- 해결 | 서버 응답을 기다리지 않는 낙관적 업데이트를 적용 및 빈 토픽으로 먼저 접속해 이벤트를 놓치던 채팅 구독 로직을 개선
- 결과 | 운영 환경과 동일한 조건에서 480회 측정한 결과 메시지 전송 후 화면 반영까지 **3,961ms → 250ms**로 개선해 체감 지연 시간 **약 93.7%** 단축

• 색인 콘텐츠가 없는 문제를 확인하고 공개 랜딩 페이지를 구현해 검색 노출 13배 개선

- 문제 | metadata와 sitemap을 설정했음에도 배포 후 7주간 검색 결과에 노출되지 않음
- 해결 | Search Console의 색인 현황을 확인해 색인 대상 페이지 자체가 없다는 원인을 발견하고 공개 랜딩 페이지 구현
- 결과 | 일평균 검색 노출을 **0.14회 → 1.90회**로 **약 13배**, 검색 노출 발생일 비율을 **14% → 81%**로 개선

• 검색어 기반 FAQ 콘텐츠를 추가해 일평균 노출 3.6배 증가

- 문제 | 랜딩 페이지 개선으로 노출은 발생했으나 최다 노출 검색어가 '넷플릭스 엔빵'일 만큼 서비스명 조합 검색에 한정되어, 새 사용자가 검색으로 유입될 검색어 폭이 좁음
- 해결 | Keyword Planner로 검색량 높은 연관 검색어를 수집해 검색 의도를 반영한 FAQ 콘텐츠를 랜딩 페이지에 추가
- 결과 | FAQ 콘텐츠 도입 이후 검색 노출 이전 4주 평균 대비 일평균 노출은 **2.2회 → 10회**로 **약 4.6배** 증가

PROJECTS

Kori / 한국에 관심 있는 외국인을 위한 소셜 커뮤니티 애플리케이션

2025.09 - 2026.02(FE 3인 / BE 3인 / 디자이너 1인)

[앱스토어 바로가기](#) → | [플레이스토어 바로가기](#) → | [깃허브 바로가기](#) →

한국 문화에 대한 새로운 소식, 퀴즈와 투표, 커뮤니티 공간, 채팅 등을 제공하는 소셜 커뮤니티 애플리케이션입니다.

1차 MVP 출시 후 프론트엔드 인력 이탈로 개발이 멈춘 상태에서 2차 MVP 개발에 새롭게 합류해 프론트엔드 리드를 맡았습니다.

누적 다운로드 300회 수준이던 초기 단계부터 **다운로드 5.7천 건, 가입자 2,648명** 규모로 성장하는 동안

푸시 알림 시스템 구현, UI/UX 개선 및 버그 대응 프로세스와 배포 체계를 정비했습니다.

TECH STACK

React Native | TypeScript | FCM | Expo | styled-components | EAS

TypeScript | 여러 명이 작업하는 환경에서 인터페이스 변경이 다른 화면에 미치는 영향을 컴파일 시점에 확인하기 위해 사용

EAS | 앱 빌드 및 배포 자동화와 OTA 업데이트를 통한 빠른 운영 대응을 위해 활용

FCM | 앱 상태, OS와 관계없이 사용자에게 실시간 푸시 알림을 전달하기 위해 선택

WHAT I DID

• 앱 알림 권한과 서버 알림 설정 동기화 구현

- **문제** | 알림 설정 동기화가 앱 최초 실행 시 1회만 실행되어 사용자가 OS 설정에서 권한을 바꿔도 서버에 반영되지 않아, 알림을 켜도 푸시가 오지 않거나 꺼도 계속 발송되는 문제 발생
- **해결** | 앱이 활성화될 때마다 OS 권한을 다시 확인하도록 AppState 기반으로 동기화 시점을 변경하고, 권한 상태에 따라 서버 알림 설정을 초기화, 비활성화, 활성화하도록 분기 처리
- **결과** | OS 설정에서 권한을 바꿔도 앱 복귀 시 서버 상태가 자동으로 맞춰져, 설정 화면과 실제 푸시 수신 여부가 어긋나는 문제 해결

• 내비게이션 스택 재구성으로 알림 진입 시 뒤로 가기 UX 개선

- **문제** | 앱 종료 상태에서 푸시 알림으로 상세 페이지에 진입하면 내비게이션 스택이 없어 뒤로 가기가 불가능한 문제 발생
- **해결** | History Back과 Hierarchy Back 두 방식을 비교해 트레이드오프를 검토한 뒤 기존 탐색 맥락을 보존하기 위해 History Back 방식을 유지하면서 알림 진입 시에만 필요한 내비게이션 스택을 먼저 구성
- **결과** | 상세 페이지의 기존 로직을 수정하지 않고도 앱 종료 후 알림으로 진입한 경우에도 일반 탐색과 동일하게 이전 화면 맥락을 유지하며 뒤로 가기가 동작하도록 개선

• EAS 기반 배포 자동화 및 OTA 업데이트 환경 구축

- **문제** | 개발자마다 코일 환경과 배포 방식이 달라 빌드가 반복적으로 실패하고 단순 수정에도 전체 앱을 다시 빌드해야 해 배포 비용과 시간 증가
- **해결** | EAS Build 기반 CI/CD와 OTA 업데이트 환경을 구축하고 PR 라벨 기반 워크플로우를 적용해 빌드와 업데이트를 목적에 맞게 분리
- **결과** | 월평균 빌드 횟수 **56% 절감** 및 월평균 빌드 실패 횟수 **5.4회 → 0.5회 감소**

• 공통 BottomSheet 컴포넌트 설계로 컴포넌트 재사용성 향상

- **문제** | 기능마다 BottomSheet를 개별 구현하면서 동일한 UI와 로직이 반복되고 수정 사항을 여러 컴포넌트에 반영해야 하는 비효율 발생
- **해결** | 다양한 화면에서 공통으로 사용할 수 있는 BottomSheet 컴포넌트를 설계하고 Props 기반으로 콘텐츠와 동작을 분리해 재사용 가능한 구조로 개선
- **결과** | 중복 코드를 줄이고 일관된 UI를 유지할 수 있는 공통 컴포넌트 구조를 구축해 유지보수성과 확장성 향상